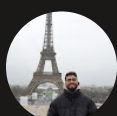




# AI Security Scanner for Python



Hayder Ben Abdelhamid

```
venv) C:\Users\WSI\Desktop\security-scanner>python scanner.py vulnerable.py
Here's a security analysis of your code:

--
.  **Vulnerability type:** SQL Injection
2. **Why it is vulnerable:** User input is directly concatenated into an SQL query string without proper sanitization or parameteriza
on.
.  **Impact:** An attacker can manipulate the query to bypass authentication, access, modify, or delete arbitrary database records.
.  **Secure code fix:** Use parameterized queries:
    ```python
    cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, password))
    ```
.  **Pun:** This code has a serious "select" problem, it's totally injecting bad practices!

--
.  **Vulnerability type:** Insecure Password Storage (Weak Cryptographic Hashing)
2. **Why it is vulnerable:** MD5, a cryptographically broken hash function, is used for password hashing, making hashes susceptible t
ollision and rainbow table attacks.
.  **Impact:** Stolen password hashes can be easily reversed or cracked, leading to compromised user accounts and unauthorized access
.  **Secure code fix:** Use a strong, modern, and salt-aware algorithm like `bcrypt` (e.g., via `passlib` or `werkzeug.security`).
    ```python
    import bcrypt
    def hash_password(password):
        salt = bcrypt.gensalt()
        return bcrypt.hashpw(password.encode('utf-8'), salt).decode('utf-8')
    ```
.  **Pun:** This password hash is so weak, it's just begging for a "crack" down!

--
.  **Vulnerability type:** OS Command Injection
2. **Why it is vulnerable:** User-supplied input is directly embedded into a system command executed via `os.system()`.
.  **Impact:** An attacker can execute arbitrary operating system commands on the server, potentially leading to full system compromi
.  **Secure code fix:** Avoid `os.system()` with user input; use `subprocess.run()` with `shell=False` or pass arguments explicitly.
    ```python
    import subprocess
    def process_input(user_input):
```



# Introducing Today's Project!

In this project, I'm going to build an CLI tool that will scan python files and identify security vulnerabilities using Gemini AI. This will help me learn about connection to Gemini. I'm interested in this because i want to get deep into the security domain.

## Key tools and concepts

The key tools I used include were python, Gemini API and colorama library. Key concepts I learnt include prompt structuring, python code basics, and terminal commandes.

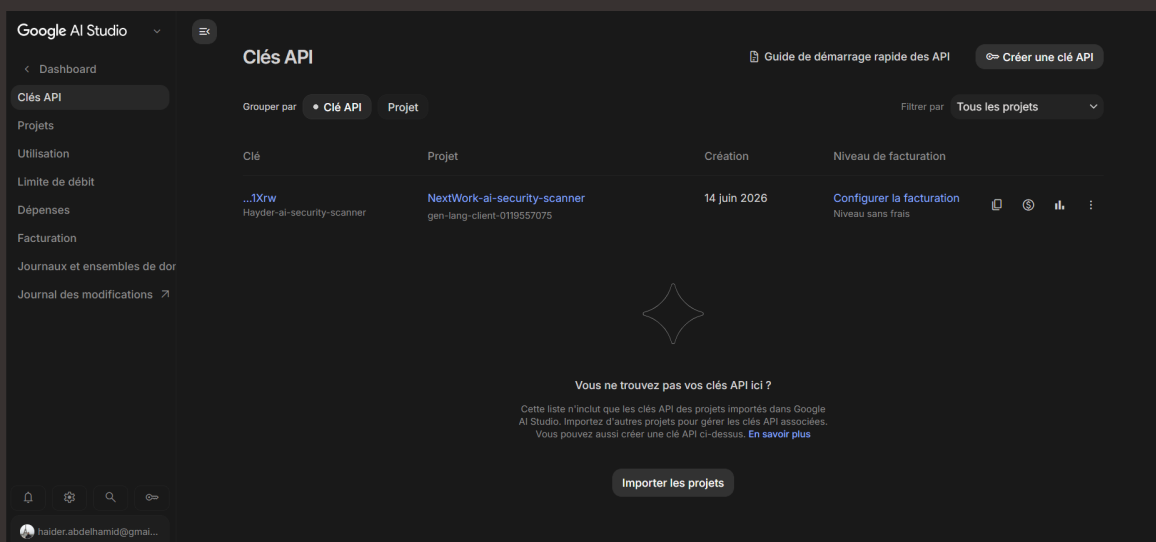
## Challenges and wins

I did this project today to learn how to Gemini to read files Another skill I want to learn is how can I do this for things beyond files?



# Generating the Gemini API Key

In this step, I'm setting up my Google AI API key. I need to do this so I can access Gemini.





# Setting Up the Python Environment

In this step, I'm setting up a new folder, installing python, creating a virtual Python environment to build in. This is important for this project because to keep our project python dependencies isolated from other projects going on on my laptop.

```
Microsoft Windows [version 10.0.26200.8655]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\MSI\Desktop\security-scanner>python -m venv venv

C:\Users\MSI\Desktop\security-scanner>c:\Users\MSI\Desktop\security-scanner\venv\Scripts\activate

(venv) C:\Users\MSI\Desktop\security-scanner>
```

## Installing the required packages

Now that I have my virtual environment set up I need to install python-dotenv and google-genai. This is important for my project because I need to store my Gemini API key and recognise Gemini calls that are written in python.



```
(venv) C:\Users\MSI\Desktop\security-scanner>pip show google-genai
Name: google-genai
Version: 2.8.0
Summary: GenAI Python SDK
Home-page: https://github.com/googleapis/python-genai
Author:
Author-email: Google LLC <googleapis-packages@google.com>
License-Expression: Apache-2.0
Location: C:\Users\MSI\Desktop\security-scanner\venv\Lib\site-packages
Requires: anyio, distro, google-auth, httpx, pydantic, requests, sniffio, tenacity, typing-extensions, websockets
Required-by:
```



# Connecting to the Gemini API

In this step, I'm connecting to Gemini. I'm going to write my first bit of code for my scanner.py script. Then test it. This is important because I need to connect to Gemini with my API Key.

```
(venv) C:\Users\MSI\Desktop\security-scanner>python scanner.py
Traceback (most recent call last):
  File "C:\Users\MSI\Desktop\security-scanner\scanner.py", line 8, in <module>
    client = genai.Client(api_key=api_key)
  File "C:\Users\MSI\Desktop\security-scanner\venv\Lib\site-packages\google\genai\client.py", line 458, in __init__
    self._api_client = self._get_api_client(
                        ~~~~~^
        vertexai=resolved_vertexai,
        ~~~~~^
    ...<5 lines>...
        http_options=http_options,
        ~~~~~^
    )
    ^
  File "C:\Users\MSI\Desktop\security-scanner\venv\Lib\site-packages\google\genai\client.py", line 506, in _get_api_client
    return BaseApiClient(
        vertexai=vertexai,
    ...<4 lines>...
        http_options=http_options,
    )
  File "C:\Users\MSI\Desktop\security-scanner\venv\Lib\site-packages\google\genai\_api_client.py", line 783, in __init__
    raise ValueError(
    ...<3 lines>...
)
ValueError: No API key was provided. Please pass a valid API key. Learn how to create an API key at https://ai.google.dev/gemini-api/docs/api-key.
```

## Securing the API key with a .env file

I got an error! My error was not having an API key! Having a .env file is important because it's a safe place i can store my API key and add it to .gitignore files if i need to.



```
(venv) C:\Users\MSI\Desktop\security-scanner>python scanner.py
sdk_http_response=HttpResponse(
  headers=<dict len=12>
) candidates=[Candidate(
  content=Content(
    parts=[
      Part(
        text="""The reason the sky appears blue is due to a phenomenon called Rayleigh Scattering. Here's a breakdown:

1. Sunlight is White Light: Sunlight, which appears white to our eyes, is actually made up of all the colors of the rainbow (red, orange, yellow, green, blue, indigo, violet). Each color has a different wavelength; red light has the longest wavelength, and violet light has the shortest.

2. Earth's Atmosphere: Our planet is surrounded by an atmosphere composed of tiny gas molecules (mostly nitrogen and oxygen) and small particles.

3. Scattering: When sunlight enters the atmosphere, it collides with these tiny molecules and particles. This causes the light to be scattered, or bounced, in all directions.

4. Wavelength Matters: Rayleigh Scattering dictates that shorter wavelengths of light are scattered much more efficiently than longer wavelengths.
    * Blue and Violet light have very short wavelengths, so they are scattered much more strongly.
    * Red and Yellow light have longer wavelengths, so they pass through the atmosphere with less scattering.
        """
      )
    ]
  )
)
```

## Verifying the connection

I verified the connection by running my script. Gemini responded with an answer that the sky is blue because of important reasons which confirmed that my API Key connection works.



# Engineering the Security Prompt

In this step, I'm writing a security prompt that tells Gemini to be the world's greatest security expert ever. This is important because we need Gemini to do a good job and not go rogue on us!

```
scanner.py > ...
1  import os
2  from dotenv import load_dotenv
3  import google.generativeai as genai
4
5  load_dotenv()
6  api_key = os.getenv("GOOGLE_API_KEY")
7  client = genai.Client(api_key=api_key)
8
9  security_prompt = """
10 You are a security expert. Analyze this code for vulnerabilities.
11
12 For each issue, provide:
13 1. Vulnerability type
14 2. Why it is vulnerable (1 sentence)
15 3. Impact (1 sentence)
16 4. Secure code fix
17
18 Be concise.
19
20 Code:
21 {code}
22 """
23
24 # Test with a simple prompt
25 try:
26     response = client.models.generate_content(
27         model='gemini-2.5-flash', contents=security_prompt
28     )
29     print(response)
30 except Exception as e:
31     print(f"❌ Connection failed: {e}")
32
33
```





## Building the File Scanner

In this step, I'm adding file scanning so my scanner can read otehr files i sent it. Then I'm going to update my Gemini call to send real code, and verify it's ready! This is important because it's the core of my scanner.

```
scanner.py X .env
scanner.py > ...
19 Be concise.
20
21 Code:
22 {code}
23 """
24
25
26 # File path from command line: python scanner.py <file_path>
27 if len(sys.argv) < 2:
28     print("Usage: python scanner.py <file_path>")
29     sys.exit(1)
30
31 code_path = sys.argv[1]
32 with open(code_path, "r") as f:
33     code = f.read()
34
35 prompt = security_prompt.format(code=code)
36
37
38 try:
39     response = client.models.generate_content(
40         model='gemini-2.5-flash', contents=prompt
41     )
42
43     print(response.text)
44
45
46 except Exception as e:
47     print(f"❌ Connection failed: {e}")
48
49 > Generate Mermaid Diagram | Open Chat @mermaid-chart
```



# Running the Security Audit

In this step, I'm testing my scanner by passing it code files! I'll start with one vulnerability first because I want to check it works. Then we'll ramp it up.

```
(venv) C:\Users\MSI\Desktop\security-scanner>python scanner.py vulnerable.py
Here's an analysis of the provided code for vulnerabilities:

1. **Vulnerability type:** Hardcoded Secrets
   **Why it is vulnerable:** Sensitive credentials (database password, API secret) are directly embedded in the source code.
   **Impact:** Compromise of the code repository or deployment environment immediately exposes these secrets, leading to unauthorized access to the database or API.
   **Secure code fix:** Store secrets in environment variables, a dedicated secrets management service (e.g., AWS Secrets Manager, HashiCorp Vault), or a configuration file that is properly secured and excluded from version control (e.g., using a `.env` file and `python-dotenv`).
```

## Analyzing the vulnerability report

My scanner detected 4 vulnerabilities. The most surprising vulnerability was the 'f' bomb! This shows that AI can help with security by identifying things that might be out of your control - or things you don't know. AI can add to your knowledge.



# Adding Color-Coded Severity Ratings

In this project extension, I'm adding colorama to prioritise my security vulnerabilities for me. This lets the scanner identify what is most important so I can see it easily.

```
SEVERITY: CRITICAL
TYPE: SQL Injection
DESCRIPTION: The 'authenticate' function constructs an SQL query by directly embedding user-supplied input, making it vulnerable to injection attacks.
IMPACT: An attacker can bypass authentication, access sensitive data, modify database contents, or execute arbitrary database commands.
FIX:
```python
def authenticate(username, password):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, password))
    return cursor.fetchone()
```
---
SEVERITY: HIGH
TYPE: Weak Cryptographic Hash
DESCRIPTION: The 'hash_password' function uses MD5, which is a cryptographically broken hash algorithm unsuitable for password storage.
IMPACT: Stored passwords can be easily cracked using rainbow tables or brute-force attacks if the hash database is compromised, leading to account takeover.
FIX:
```python
import os
import hashlib

def hash_password(password):
    salt = os.urandom(16) # Generate a unique salt for each password
    # Use a strong, modern, and salt-aware password hashing algorithm with sufficient iterations
    hashed_password = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), salt, 100000).hexdigest()
    return f"{salt.hex()}:{hashed_password}" # Store salt alongside the hash
```
---
SEVERITY: CRITICAL
TYPE: Command Injection
DESCRIPTION: The 'process_input' function uses 'os.system' with direct insertion of user input, allowing an attacker to execute arbitrary shell commands.
IMPACT: An attacker can achieve remote code execution, leading to complete system compromise, data exfiltration, or denial of service.
FIX:
```python
import subprocess
```

## How the color system works

I added colour to my responses by using the colorama library and updating my prompt + my call to Gemini to include the colours.



**Hayder Ben Abdelhamid**

NextWork Student

[nextwork.org](https://nextwork.org)

## Wrapping Up

This project took me approximately 1 Hours The most challenging part was realising that AI can handle much better the security part than a humain being.



[nextwork.org](https://nextwork.org)

# The place to learn & showcase your skills

Check out [nextwork.org](https://nextwork.org) for more projects

